are-self: A Revolutionary Agentic Brain for Graph-Based Orchestration and Autonomous Reasoning

are-self Project

March 18, 2026

Abstract

are-self is an AI-assisted automation platform that unifies two traditionally separate domains: graph-based workflow orchestration and identity-driven autonomous reasoning. Built on the **are-self** Django control plane, it addresses the industry pain of fragmented automation tools by providing a single system where build pipelines, remote agent execution, and AI reasoning run as first-class citizens of the same graph vocabulary. The platform delivers persistent, typed state for full audit trails and recovery; a formal transition semantics for labeled directed graphs; and a cognition stack with focus/XP economy, vector memory, and a work-eligibility gatekeeper that prevents spinning up AI workers without actionable work. This whitepaper provides exhaustive technical detail for lead developers, data scientists, and machine learning engineers: architecture diagrams, methodology breakdowns, code snippets from the live codebase, and verified implementation parameters.

1 Problem Statement

The development of AI-assisted automation has reached a critical juncture: orchestration and cognition are typically implemented as separate systems, leading to fragmented workflows, lost context, and inefficient resource use.

1.1 Orchestration and Cognition as Separate Concerns

Traditional automation platforms treat workflow orchestration (launching builds, running scripts, distributing work across agents) and AI reasoning (LLM turn loops, tool use, memory) as distinct domains. This split forces engineers to glue together disparate tools, lose execution context when crossing boundaries, and accept that AI workers cannot natively participate in the same graph that drives process execution. The result is brittle pipelines, manual handoffs, and systems that cannot recover gracefully when AI sessions fail or time out [1].

1.2 The Need for Persistent, Typed State

Many orchestration systems rely on transient in-memory state. When a process crashes, a network partition occurs, or a Celery worker restarts, execution context is lost. Audit trails are incomplete, and recovery requires manual intervention. In AI-assisted workflows, this is especially problematic: a reasoning session that has consumed dozens of turns and tool calls cannot be resumed if its state was never persisted. The industry needs systems where almost every important runtime decision is driven by database rows, enabling inspectable state, full audit trails, and recovery from failures without losing execution context [1].

1.3 Scaling and Resource Efficiency

Distributed execution across local and remote agents introduces complexity: discovery, health checks, log streaming, and failure handling must be unified. AI workers, meanwhile, consume significant compute; spinning up sessions when no actionable work exists wastes resources and increases cost. Systems need a work-eligibility gatekeeper that decides *whether* an AI should work before creating a session, and a scheduler that decides *when* it could work—avoiding idle cognition and ensuring agents only run when they have real tasks [4, 5].

2 The Solution & Architecture

are-self is implemented by **are-self**, a Django control plane that couples a graph-based orchestration engine with an autonomous reasoning engine. Both halves share the same vocabulary: a reasoning run is another kind of spike execution, invoked via the native handler `run_frontal_lobe`. The system is built around persistent, typed state and a brain-themed modular decomposition.

2.1 System Design Overview

The architecture has two pillars: (i) *Orchestration*—the Central Nervous System (CNS) graph engine, Peripheral Nervous System (PNS) agent discovery and execution, and environments for context—and (ii) *Cognition*—frontal lobe (reasoning), parietal lobe (tools), hippocampus (memory), temporal lobe (scheduling), prefrontal cortex (work allocation), and identity (persona). The CNS connects to PNS and environments; the CNS invokes frontal lobe and temporal lobe; the frontal lobe connects to parietal lobe, hippocampus, and identity; the temporal lobe connects to prefrontal cortex; and the prefrontal cortex invokes the frontal lobe when work exists [2, 1].

2.2 Request Path and URL Routing

The live runtime uses the `central_nervous_system` stack registered in `config/settings.py` and routed from `config/urls.py`. The active request path is [1]:

Listing 1: URL routing and API registration (config/urls.py).

```
urlpatterns = [
path("", include("dashboard.urls")),
path("central_nervous_system/", include("central_nervous_system.urls.urls")),
path("environments/", include("environments.urls")),
path("reasoning/", include("frontal_lobe.urls")),
path("api/v1/", include(v1_router.urls)),
path("api/v2/", include(V2_ROUTER.urls)),
path("admin/", admin.site.urls),
path("api-auth/", include("rest_framework.urls", namespace="rest_framework")),
path("mcp/", include("djangorestframework_mcp.urls")),
]
```

V2 routers extend CNS, Temporal Lobe, Identity, Prefrontal Cortex, Hippocampus, Reasoning, Parietal Lobe, and PNS. Note: there are two distinct "MCP" concepts—external `djangorestframework_mcp` at `/mcp/` (DRF viewsets) and internal `ParietalMCP.execute()` which dynamically loads `mcp_*` tool modules for the LLM [1].

2.3 Architecture Diagram: High-Level Data Flow

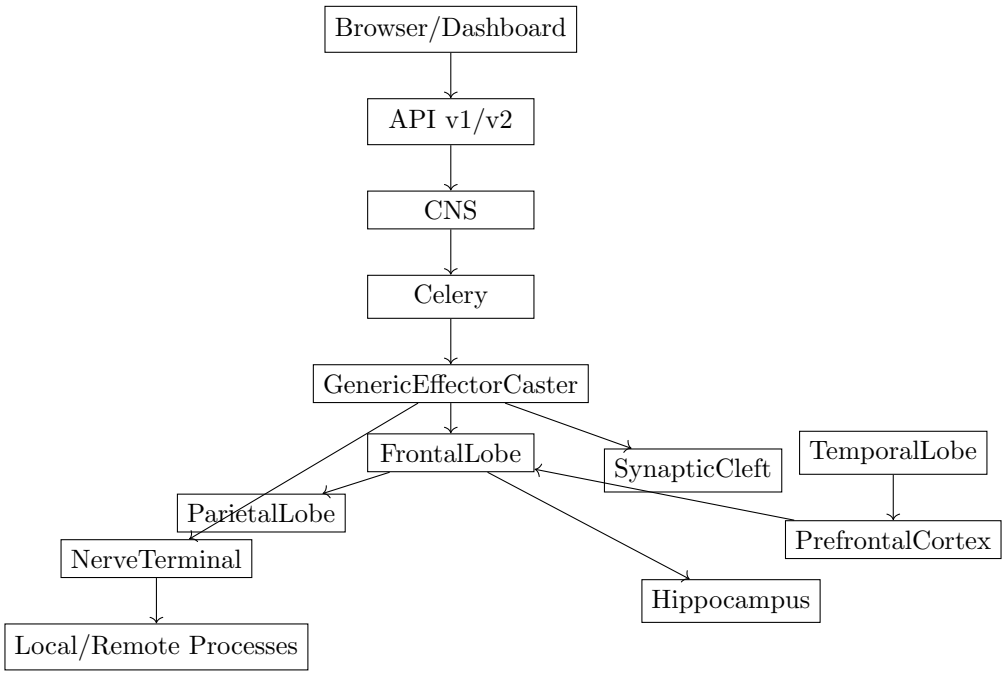

Figure 1: High-level data flow: CNS routes through Celery to GenericEffectorCaster, which dispatches to NerveTerminal (processes), FrontalLobe (reasoning), or SynapticCleft (events). TemporalLobe → PrefrontalCortex → FrontalLobe when work exists.

2.4 Cross-Module Architecture (Orchestration and Cognition)

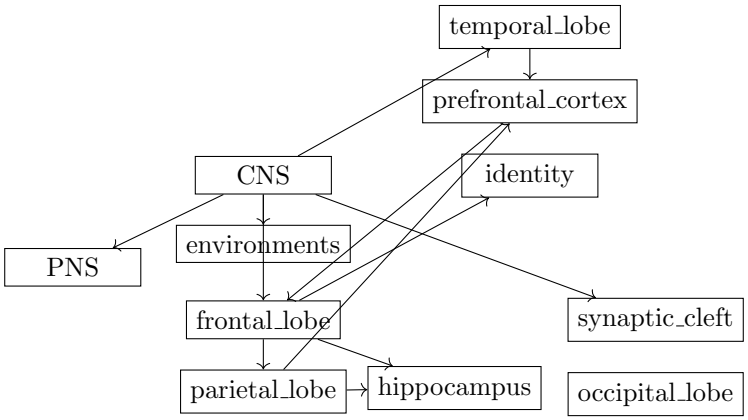

Figure 2: Cross-module architecture from `docs/doc-generator/combined__documentation.md`.

2.5 Graph Vocabulary and Mathematical Model

The static graph is [3, 2, 1]:

- **NeuralPathway:** the protocol or graph container.

- **Neuron:** a node in that graph.
- **Axon:** a directed edge between two neurons, labeled as `flow`, `success`, or `failure`.
- **Effector:** a configured action bound to a `are-selfExecutable`, argument assignments, switches, and a distribution mode.

The runtime graph is:

- **SpikeTrain:** one execution of a `NeuralPathway`.
- **Spike:** one execution of one `Neuron`/`Effector` within that `SpikeTrain`.
- **provenance:** the parent `Spike` that caused a later `Spike` to exist.
- **blackboard:** JSON working memory carried along execution branches.

Mathematically, the static graph is $G = (V, E, \lambda)$ where $V = \{\text{Neuron}\}$, $E \subseteq V \times V$, and $\lambda(e) \in \{\text{flow}, \text{success}, \text{failure}\}$. The runtime is an unfolding execution tree over time, not a simple replay of G [3, 1].

2.6 Transition Rule and Wave Dispatch

For finished spike s with $\sigma(s) \in \{\text{SUCCESS}, \text{FAILED}\}$ [3, 2]:

$$L_{\text{enabled}}(s) = \{\text{flow}\} \cup \begin{cases} \{\text{success}\} & \sigma(s) = \text{SUCCESS} \\ \{\text{failure}\} & \sigma(s) = \text{FAILED} \end{cases}$$

For each $e = (\nu(s), v)$ with $\lambda(e) \in L_{\text{enabled}}(s)$: create spike at v. The `cast_cns_spell` Celery task executes one spike; in `finally`, it always calls `check_next_wave`, making the engine self-driving.

2.6.1 Implementation: dispatch_next_wave

The core dispatch logic from `central_nervous_system/central_nervous_system.py` [1]:

Listing 2: Wave dispatch logic (simplified).

```
def dispatch_next_wave(self) -> None:
with transaction.atomic():
self.spike_train.refresh_from_db()
if self.spike_train.status_id == SpikeTrainStatus.STOPPING
    :
self._finalize_spawn_unsafe()
return
spikes = self.spike_train.spikes.select_for_update().all()
if not spikes.exists():
self._dispatch_graph_roots()
return
finished_spikes = spikes.filter(
status_id__in=[SpikeStatus.SUCCESS, SpikeStatus.FAILED]
)
parents_with_children = Spike.objects.filter(
spike_train=self.spike_train, provenance__isnull=False
).values_list("provenance_id", flat=True)
for spike in finished_spikes:
```

```
                if spike.id in parents_with_children:
                continue
                self._process_graph_triggers(spike)
                self._finalize_spawn_unsafe()

                def _process_graph_triggers(self, finished_spike: Spike) -> None:
                valid_wire_types = [AxonType.TYPE_FLOW]
                if finished_spike.status_id == SpikeStatus.SUCCESS:
                valid_wire_types.append(AxonType.TYPE_SUCCESS)
                elif finished_spike.status_id == SpikeStatus.FAILED:
                valid_wire_types.append(AxonType.TYPE_FAILURE)
                axons = Axon.objects.filter(
                pathway=self.spike_train.pathway,
                source=finished_spike.neuron,
                type_id__in=valid_wire_types,
                )
                for wire in axons:
                self._create_spike_from_node(
                neuron=wire.target, provenance=finished_spike
                )
```

2.6.2 Self-Driving Loop: cast_cns_spell

Listing 3: Celery task `cast_cns_spell` (central_nervous_system/tasks.py).

```
                @shared_task(bind=True)
                def cast_cns_spell(self, spike_id):
                spike_train_id = None
                try:
                spike = Spike.objects.only("spike_train_id").get(id=spike_id)
                spike_train_id = spike.spike_train_id
                spike.celery_task_id = self.request.id
                spike.save(update_fields=["celery_task_id"])
                caster = GenericEffectorCaster(spike_id=spike_id)
                caster.execute()
                except Exception as e:
                spike = Spike.objects.get(id=spike_id)
                spike.status_id = SpikeStatus.FAILED
                spike.execution_log += f"\n[CELERY FATAL] Task crashed: {e}\n"
                spike.save(update_fields=["status", "execution_log"])
                raise e
                finally:
                if spike_train_id:
                transaction.on_commit(lambda: check_next_wave.delay(spike_train_id))
```

The `finally` block ensures the next wave is always triggered; that is the self-driving property [1].

2.7 Context Resolution Precedence

Context composition: $C = \text{metadata} \oplus \text{env} \oplus \text{blackboard} \oplus \text{effector} \oplus \text{neuron}$. Precedence (rightmost wins): neuron > effector > blackboard > env > metadata [3, 2]. The implementation in `resolve_environment_context` applies: metadata/base objects, then environment variables, then blackboard, then effector defaults, then neuron overrides [1].

2.8 Distribution Modes

After spike creation, cardinality is chosen [3, 2]:

- `LOCAL_SERVER`: one spike, no remote target.
- `ONE_AVAILABLE_AGENT`: one spike, first online `NerveTerminalRegistry` by `last_seen`.
- `ALL_ONLINE_AGENTS`: clone one spike per online agent.
- `SPECIFIC_TARGETS`: clone one spike per pinned `EffectorTarget`.

2.9 Cognition Engine: Session Model and Turn Loop

The cognitive runtime is centered on `ReasoningSession`, `ReasoningTurn`, `ChatMessage`, `SessionConclusion`, `ToolDefinition/ToolCall`, and `are-selfEngram`. A `ReasoningSession` is linked to its originating `Spike` and optionally to `IterationShiftParticipant`. `IdentityDisc` provides persona, enabled tools, and model [1].

Each turn assembles from: (1) system identity prompt, (2) L1/L2 history cache, (3) volatile addon messages, (4) sensory trigger (engram catalog). L1: last 2 turns full; L2: older turns with truncated tool results and pruned monologue [6, 1].

2.10 Session Economy (Frontal Lobe)

[2, 6]

$$\text{current_level} = \lfloor \text{total_xp}/100 \rfloor + 1, \quad \text{max_focus} = 10 + \lfloor (\text{current_level} - 1) \cdot 0.5 \rfloor$$

Efficiency bonus: if $\text{len}(\text{thought_process}) \leq \text{current_level} \cdot 1000$, grant +1 focus and +5 XP.

2.11 Tool Economy (Parietal Lobe)

Fizzle: $\Delta f < 0 \wedge f + \Delta f < 0 \Rightarrow$ tool not executed. Update: $f' = \min(\text{max_focus}, f + \Delta f)$; $\text{total_xp}' = \text{total_xp} + \text{xp_gain}$. Tool calls are sorted by `focus_modifier` descending [7, 2].

2.12 Hippocampus: Vector Memory and Novelty

Embedding $\phi : \mathcal{T} \to \mathbb{R}^{768}$; similarity $\text{sim}(\mathbf{a}, \mathbf{b}) = 1 - d_{\cos}(\mathbf{a}, \mathbf{b})$. Intercept: $\text{max_sim} \geq 0.90 \Rightarrow$ save rejected. Novelty: $\text{novelty} = \max(0, 1 - \text{similarity})$. Rewards: $\text{focus_yield} = \max(1, \lfloor 10 \cdot \text{novelty} \rfloor)$, $\text{xp_yield} = \max(5, \lfloor 100 \cdot \text{novelty} \rfloor)$ [8, 2]. Embeddings use `nomic-embed-text` via Ollama [8].

2.13 Temporal Lobe and Prefrontal Cortex

Temporal Lobe: iteration state WAITING → RUNNING → FINISHED; participant SELECTED → ACTIVATED → COMPLETED. Prefrontal Cortex: $W(\text{shift}, \text{identity_type}, \text{disc}, \text{env})$ is a finite table. If no work, worker is stood down to SELECTED. **TemporalLobe** decides when an AI could work; **PrefrontalCortex** decides whether it should work [4, 5, 2].

2.14 Internal Tool Families (ParietalMCP)

The internal `ParietalMCP` dynamically imports modules named `mcp_*` [1]:

- Backlog/work: `mcp_ticket` (PFCEpic, PFCStory, PFCTask).
- Memory: `mcp_engram_save`, `mcp_engram_read`, `mcp_engram_search`, `mcp_engram_update`.
- State: `mcp_update_blackboard`, `mcp_pass`, `mcp_done`.
- Cognitive: `mcp_internal_monologue`.
- File/repo: `mcp_read_file`, `mcp_list_files`, `mcp_grep`, `mcp_fs`, `mcp_git`.
- Inspection: `mcp_query_model`, `mcp_read_record_field`, `mcp_search_record_field`, `mcp_inspect_record`.
- Browser/internet: `mcp_browser_read`, `mcp_internet_query`.

2.15 State Machines

Table 1 summarizes key state machines from [2].

Table 1: **are-self** state machines.

Entity	**States**	**Terminal**
Spike	CREATED, PENDING, RUNNING, SUCCESS, FAILED, ABORTED, DELEGATED, STOPPING, STOPPED	SUCCESS, FAILED, ABORTED, STOPPED
SpikeTrain	CREATED, RUNNING, SUCCESS, FAILED, STOPPING, STOPPED	SUCCESS, FAILED, STOPPED
ReasoningSession	PENDING, ACTIVE, PAUSED, COMPLETED, MAXED_OUT, ERROR, ATTENTION_REQUIRED, STOPPED	COMPLETED, MAXED_OUT, ERROR, STOPPED
Iteration	WAITING, RUNNING, FINISHED, CANCELLED, BLOCKED_BY_USER, ERROR	FINISHED, etc.
IterationShiftParticipant	SELECTED, ACTIVATED, COMPLETED, PAUSED, ERROR	COMPLETED, etc.

3 Implementation & Benchmarks

are-self is implemented in the **are-self** codebase as a Django application with Celery, Redis, and PostgreSQL/SQLite. The live runtime uses the `central_nervous_system` stack plus brain-themed apps.

3.1 Implementation Parameters (Verified from Codebase)

Table 2 lists verified constants and thresholds from the codebase and documentation. No performance benchmarks (latency, throughput) are published; the table documents architectural and configuration parameters only [1, 2, 3, 6, 4, 8, 9, 10].

Table 2: Verified implementation parameters from codebase and docs.

Parameter	Value	Source
STALE_PENDING_TIMEOUT	5 min	`central_nervous_system.py`
ZOMBIE_THRESHOLD_MINUTES	20	`temporal_lobe.py`
DEFAULT_MAX_TURNS	100	`frontal_lobe/constants.py`
max_concurrent_workers	1	`temporal_lobe.py`
Vector dimensions	768	`hippocampus/models.py`
Hippocampus intercept	0.90	`hippocampus/hippocampus.py`
Dashboard polling overlap	2.5 s	`dashboard/api.py`
Agent discovery probe	1.5 s	`peripheral_nervous_system`
Occipital error blocks	5 before, 10 after; max 5	`occipital_lobe/readers.py`
Occipital max_char_limit	$(\text{max_token_budget} - 2000) \times 4$	`combined__documentation.md`
Prompt-window heuristic	$\lfloor \text{payload_size}/3 \rfloor + 2048$	`frontal_lobe/synapse.py`

3.2 Architectural Characteristics

Table 3 summarizes architectural characteristics of **are-self** vs. fragmented orchestration + AI, derived from the design (not from performance benchmarks).

Table 3: Architectural characteristics (design-derived, not benchmarked).

Characteristic	are-self	Notes
State persistence	Full	All decisions driven by DB rows [1]
Recovery context	Preserved	Blackboard, session, turn state persisted
AI work gating	PFC gate	No session without `_is_available_work` [5]
Unified vocabulary	Yes	Spike/SpikeTrain for process and reasoning
Wave dispatch	Self-driving	`check_next_wave` in Celery `finally`

3.3 Case Study

A representative workflow: launch a NeuralPathway that (1) runs a build step via NerveTerminal, (2) on success invokes `run_frontal_lobe` for an AI refinement step, (3) the AI uses `mcp_ticket` and `mcp_engram_save` tools. The entire flow is one SpikeTrain; the AI spike is a first-class graph node. If the build fails, the failure edge fires and alternative nodes execute; if the AI maxes out turns, the session concludes and the graph advances.

4 Conclusion & Call to Action

are-self demonstrates that orchestration and cognition can be unified in a single platform. The **are-self** implementation provides persistent state, formal graph semantics, and a cognition stack with economy-based resource management and work gating.

4.1 Summary

The industry suffers from fragmented automation: orchestration and AI reasoning are separate, state is often transient, and AI workers run without work-eligibility checks. **are-self** addresses these by coupling graph-based orchestration with identity-driven reasoning in one Django control plane, using the same graph vocabulary for both, persisting all critical state, and gating AI sessions on the presence of actionable work.

4.2 Call to Action

- Explore the **are-self** codebase: `docs/ARCHITECTURE.md`, `docs/LIVE_CODEBASE_ANALYSIS.md`, `docs/doc-generator/`.
- Review the API: REST at `api/v1/` and `api/v2/`, MCP at `/mcp/`.
- Run the Mission Control dashboard to launch pathways and observe spike execution.
- Contact the **are-self** project for contributions, enterprise support, or integration guidance.

References

[1] are-self Live Codebase Analysis. `docs/LIVE_CODEBASE_ANALYSIS.md`.

[2] are-self Backend Combined Mathematical Structure. `docs/doc-generator/combined__documentation.md`.

[3] Central Nervous System Documentation. `docs/doc-generator/central_nervous_system__documentation.md`.

[4] Temporal Lobe Documentation. `docs/doc-generator/temporal_lobe__documentation.md`.

[5] Prefrontal Cortex Documentation. `docs/doc-generator/prefrontal_cortex__documentation.md`.

[6] Frontal Lobe Documentation. `docs/doc-generator/frontal_lobe__documentation.md`.

[7] Parietal Lobe Documentation. `docs/doc-generator/parietal_lobe__documentation.md`.

[8] Hippocampus Documentation. `docs/doc-generator/hippocampus__documentation.md`.

[9] Occipital Lobe Documentation. `docs/doc-generator/occipital_lobe__documentation.md`.

[10] Peripheral Nervous System Documentation. `docs/doc-generator/peripheral_nervous_system__documentation.md`.

A Appendix A: CNS Transition Rule (Formal)

For finished spike s with $\sigma(s) \in \{\text{SUCCESS}, \text{FAILED}\}$:

$$L_{\text{enabled}}(s) = \{\text{flow}\} \cup \begin{cases} \{\text{success}\} & \sigma(s) = \text{SUCCESS} \\ \{\text{failure}\} & \sigma(s) = \text{FAILED} \end{cases}$$

For each $e = (\nu(s), v)$ with $\lambda(e) \in L_{\text{enabled}}(s)$: create spike at v [3, 2].

B Appendix B: Hippocampus Novelty and Rewards

When save is not intercepted:

$$\text{novelty} = \max(0, 1-\text{similarity}), \quad \text{focus_yield} = \max\left(1, \lfloor 10 \cdot \text{novelty} \rfloor\right), \quad \text{xp_yield} = \max\left(5, \lfloor 100 \cdot \text{novelty} \rfloor\right)$$

[8, 2]

C Appendix C: Invariants (from combined documentation)

1. CNS: at most one Begin Play neuron per pathway; `cast_cns_spell` always calls `check_next_wave` in `finally`.
2. Environments: at most one `ProjectEnvironment` with `selected=True`.
3. Hippocampus: vector dim = 768; intercept threshold = 0.90.
4. PFC: no `ReasoningSession` without `_is_available_work` True.
5. Temporal: at most one active temporal `SpikeTrain` per environment.

[2]